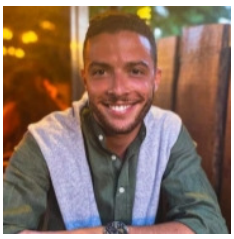
Aplicação de Serviços de Computação em Nuvem - Trabalho Prático

Trabalho realizado por:

Francisco Alves Andrade PG47185
João Figueiredo Martins Peixe dos Santos PG47304
Luís Carlos Sousa Magalhães 47415
Paulo Silva Sousa PG47556
Tiago Carvalho Freitas PG47687



PG47185
Francisco Andrade

PG47304
João Santos

PG47415
Luís Magalhães

PG47556
Paulo Sousa

PG47687
Tiago Freitas

Grupo 3
Universidade do Minho

Conteúdo

1 Introdução **1**

2 Tarefa 1 **2**
2.1 Arquititetura e componentes da aplicação 2
2.2 Identificação dos padrões de distribuição da aplicação e dos diferentes componentes 2
2.3 Identificação de componentes e operações cujo desempenho é crítico 2

3 Tarefa 2 **3**
3.1 Estrutura e relação entre máquinas virtuais 3
3.2 Ansible 4
3.3 Instalação de compontentes diferentes em containers independentes 5
3.4 Utilização de serviços extra da *Google Cloud Platform* 5
3.5 Componentes que devem ser replicados 5

4 Tarefa 3 **6**
4.1 MetricBeat 6
4.2 ElasticSearch 6
4.3 Kibana 6
4.4 Métricas Utilizadas 7

5 Tarefa 4 **8**
5.1 Apache JMeter 8
5.2 Testes e resultados 8
5.2.1 Thread = 10 9
5.2.2 Thread = 100 9
5.2.3 Thread = 500 9
5.2.4 Thread = 1000 10
5.3 Análise dos Resultados 10

6 Conclusão **10**

1 Introdução

Este relatório aborda o que foi desenvolvido no âmbito do trabalho prático da Unidade Curricular de Aplicação de Serviços de Computação em Nuvem, que consistiu na caracterização, análise e na automatização da instalação, monitorização e avaliação da aplicação Wiki.js, num ambiente de computação em nuvem (Google Cloud Plataform) aplicando os conceitos lecionados nas aulas desta unidade.

O grupo optou por seguir as etapas que foram sugeridas no enunciado, conseguindo, assim, manter a organização e ter ideia do que ainda faltava implementar. Ao longo deste relatório iremos explicitar por completo o desenvolvimento, assim como esclarecer todas as decisões tomadas durante as diversas etapas até ao resultado final e quais as conclusões que foram tiradas após a realização deste projeto.

2 Tarefa 1

2.1 Arquititetura e componentes da aplicação

A arquitetura usada para a aplicação é a *multi-tier*, que garante flexibilidade, isto é, faz com que seja possível desenvolver e/ou alterar funcionalidades de forma independente, uma vez que se trabalha apenas com o respetivo *layer*, em vez de refazer a aplicação inteira.

Como componentes temos **Wiki.js** e a sua **Base de Dados**. Utilizando a arquitetura *multi-tier* obtemos uma representação da aplicação com eficácia e exatidão.

2.2 Identificação dos padrões de distribuição da aplicação e dos diferentes componentes

Figura 1: Arquitetura *multi-tier* da aplicação

2.3 Identificação de componentes e operações cujo desempenho é crítico

Para obter um bom desempenho existem operações cujo desempenho é crítico. Podemos considerar operações críticas as seguintes:

- **Isolamento dos componentes em containers diferentes:** Proporcionará maior robustex e mais disponibilidade à aplicação.
- **Replicação de componentes:** Com a replicação deixará de existir apenas um ponto de falha e será possível um amaior disponibilidade.
- **Conexão entre os componentes da aplicação:** Caso esta conexão não existisse a aplicação não seria funcional.

3 Tarefa 2

3.1 Estrutura e relação entre máquinas virtuais

Nesta tarefa utilizamos o Ansible para instalar de forma automática e configurável os diversos compentes da aplicação num ambiente de computação em núvem. Para isso, utilizamos o *Google Cloud Platform*.

Assim, decidimos criar 4 máquinas virtuais da série *e2-standard-2* (2vCPU, 8GB memória) como podemos verificar no seguinte esquema:

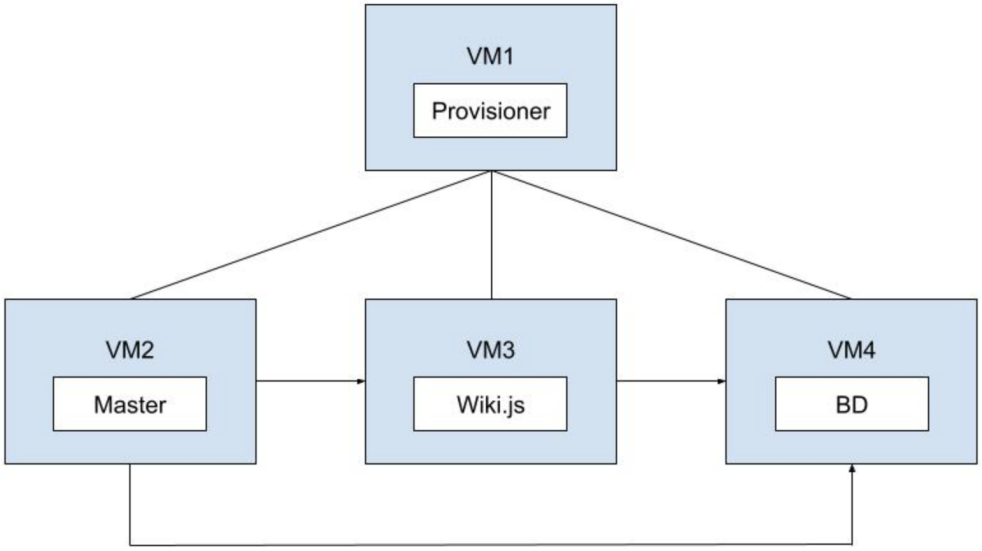


Figura 2: Esquema da Estrutura da Aplicação

Status	Name	Zone	Recommendations	In use by	Internal IP	External IP	Connect
	instance-1	europe-west1-b			10.132.0.5 (nic0)	34.77.201.4	SSH
	instance-2	europe-west1-b			10.132.0.21 (nic0)	34.79.240.215	SSH
	instance-3	europe-west1-b			10.132.0.22 (nic0)	35.195.202.137	SSH
	instance-4	europe-west1-b			10.132.0.23 (nic0)	35.195.231.179	SSH

Figura 3: Máquinas Virtuais criadas na *Google Cloud Platform*

Como podemos ver pelo esquema acima, cada uma das máquinas virtuais vai ter uma função diferente na estrutura da aplicação:

- **VM1 -** Máquina virtual que desempenha o papel de *Provisioner*, pois é nesta máquina que irão correr os playbooks de *Ansible*, responsáveis pela instalação automática de todos os componetes da aplicação.
- **VM2 -** Máquina virtual que desempenha o papel de *Master* de *Kubernets*, pois é responsável pelo deployment e controlo de nodes e pods deste.
- **VM3 -** Máquina virtual que desempenham o papel de *Workers* do *Kubernets*, pois é nesta máquina que irão correr os dois nodes necessários para a aplicação *Wiki.js* e para a base de dados.

3.2 Ansible

Para a realização deste projeto, a ferramenta escolhida pelo grupo foi o *Ansible*, pois foi a ferramenta utilizada nas aulas.

O projeto foi dividido da seguinte forma:

- **hosts -** Ficheiro inventário
- **Install.yml -** Playbook para instalação
- **CreateClusters.yml -** Playbook para criar os clusters
- **InstallMonitorize.yml -** Playbook para instalação dos pacotes de monitorização
- **Wikijs.yml -** Playbook para dar deploy à aplicação *Wiki.js*
- **PostegreSQL.yml -** Playbook para dar deploy à base de dados
- **roles -** Desenvolvemos 7 roles diferentes

Vamos agora explicar o objetivo de cada role:

- **common** - Responsável por instalar os componentes do kubernets e docker da aplicação
- **database** - Responsável por dar deploy à base de dados *PostegreSQL*
- **installation_master** - Responsável por instalar os componentes do kibana e do elastic search
- **installation_workers** - Responsável por instalar os componentes do metricbeats
- **setup_master** - Responsável por cria o cluster
- **wikijs** - Responsável por dar deploy à aplicação *Wiki.js*
- **setup_workers** - Responsável por associar os workers ao cluster

3.3 Instalação de compontentes diferentes em containers independentes

De maneira a tirar partido da utilização de containers, instalamos a aplicação *Wiki.js* e a base de dados *PostgreSQL* em dois containers diferentes. Assim, conseguimos extrair uma melhor performance da solução, permitindo usufruir de todos os recursos disponíveis. Além disso, garantimos também que se um dos componentes for abaixo, o cluster mantém-se em funcionamento.

3.4 Utilização de serviços extra da *Google Cloud Platform*

Embora ter nos sido dada a possibilidade de utilizar os serviços extra da *Google Cloud Platform*, como bases de dados e ferramentas de monitorização, decidimos utilizar as ferramentas utilizadas nas aulas da Unidade Curricular, pois estamos mais familiarizados com estas.

3.5 Componentes que devem ser replicados

Componentes como a aplicação *Wiki.js* e a base de dados *PostgreSQL* poderiam ter sido replicadas, de maneira a não termos um único ponto de falha. No entanto, não efetuamos nenhuma réplica na nossa aplicação, mesmo conhecendo as vantagens que iria trazer para a fiabilidade e robustez do mesmo.

4 Tarefa 3

Nesta fase, o foco principal foi realizar a monotorização da aplicação, observando e analisando o seu comportamento em ambiente de teste. Para isso, incluímos na aplicação as ferramentas que irão ser abordadas em baixo.

Também foi decidido que se iria usar os recursos físicos CPU e memória RAM como métricas, pois possuem capacidade limitada, são de fácil análise e permitem que se tirem conclusões de uma forma direta.

4.1 MetricBeat

MetricBeat é uma ferramenta de apoio à monitorização, pois recolhe periodicamente métricas do sistema operativo e dos serviços em execução no servidor em questão. O *MetricBeat* pega nas métricas e estatísticas que recolhe e envia-as, neste caso, para o *ElasticSearch*, que, como iremos falar posteriormente, é responsável por armazenar esta informação.

4.2 ElasticSearch

O *ElasticSearch* é um mecanismo de procura e análise de dados distribuído, capaz de atender a um número crescente de casos de uso. Esta ferramenta foi escolhida por conseguir armazenar grandes quantidades de dados centralmente o que torna a sua procura e análise processos extremamente rápidos, assim como a obtenção de respostas.

4.3 Kibana

Kibana consiste numa *dashboard*, que permite a visualização, análise e procura dos dados indexados no *ElasticSearch*, basicamente representa a interface gráfica deste último. Fornece inúmeras opções de gestão e análise como gráficos, tabelas e histogramas.

4.4 Métricas Utilizadas

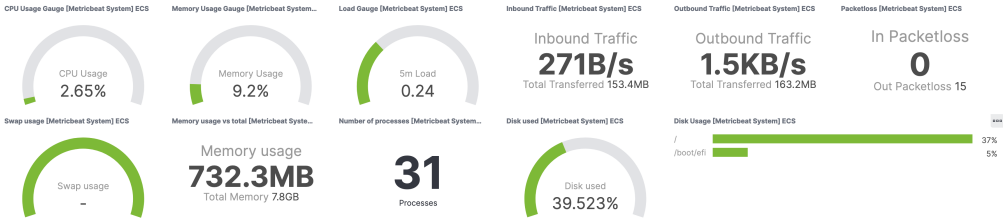


Figura 4: Visão geral da utilização de recursos

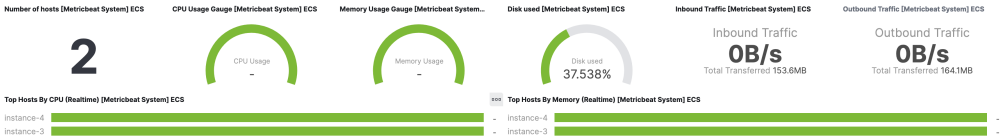


Figura 5: Visão da utilização de recursos da VM3

Figura 6: Visão da utilização de recursos da VM3

5 Tarefa 4

5.1 Apache JMeter

Apache JMeter é uma ferramenta de avaliação do *Apache*, tal como o nome indica e devido à sua utilização nas aulas práticas já estamos familiarizados com ela. Através da realização de testes consegue medir o desempenho de aplicações. Desta forma demonstramos em seguida a arquitetura e relação entre a ferramenta de avaliação (*JMeter*) e as ferramentas de monitorização (*Kibana*, *MetricBeat* e *ElasticSearch*).

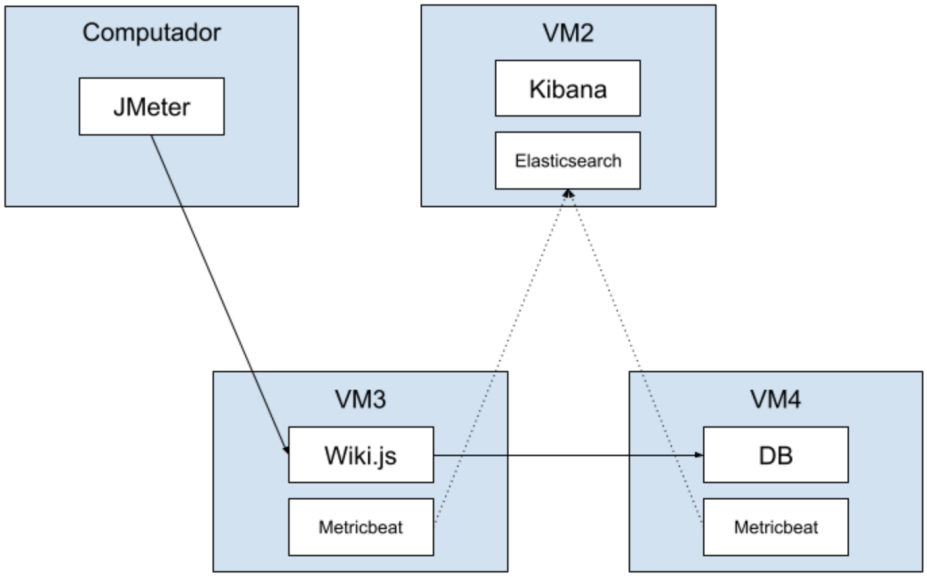


Figura 7: Monitorização

5.2 Testes e resultados

De modo a obter uma melhor percepção sobre a aplicação e a quantidade de recursos que ela utiliza decidimos realizar testes para 10, 100, 500 e 1000 threads (simulando, desta forma, utilizadores), a realizarem GET requests e a correrem num loop infinito. Assim, conseguimos testar a sobrecarga da aplicação.

5.2.1 Thread = 10

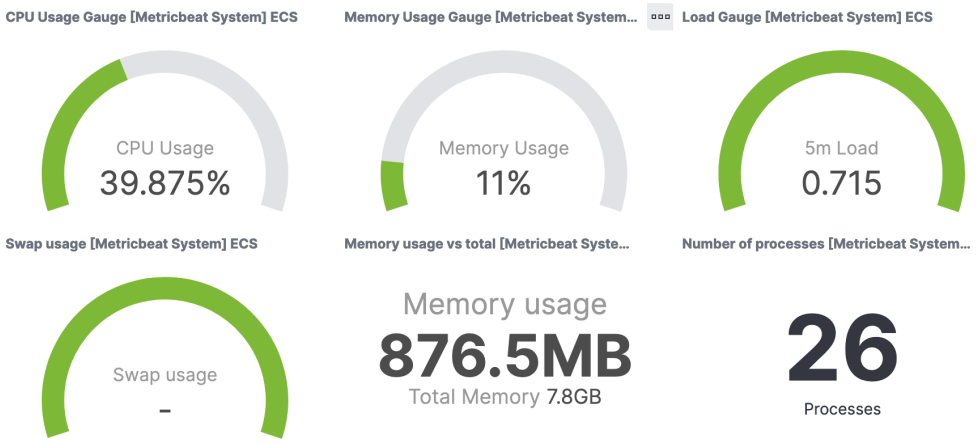


Figura 8: Teste para 10 threads

5.2.2 Thread = 100

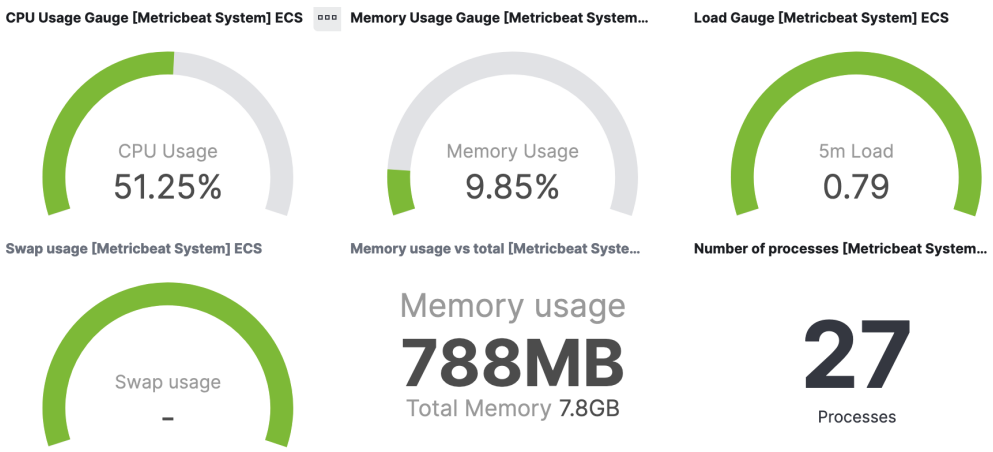


Figura 9: Teste para 100 threads

5.2.3 Thread = 500

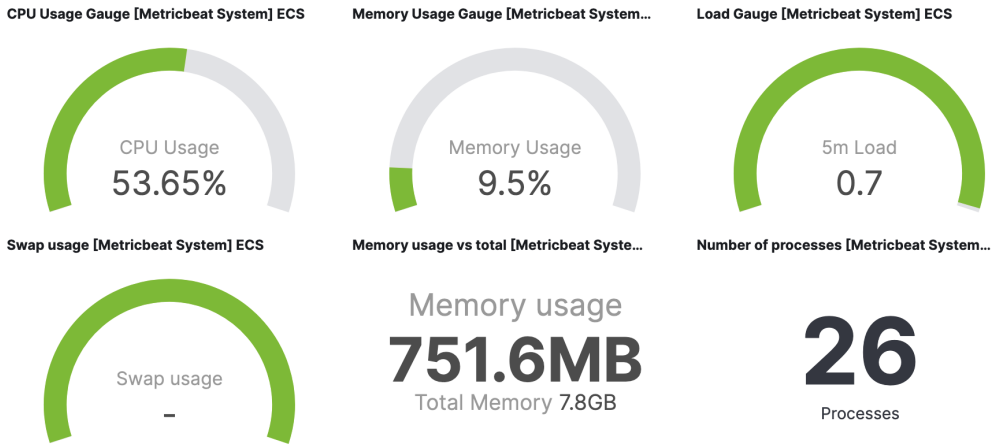


Figura 10: Teste para 500 threads

5.2.4 Thread = 1000

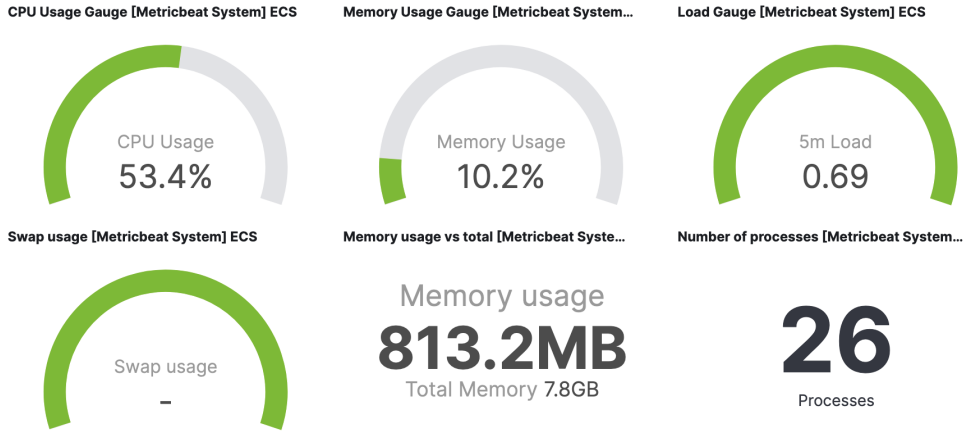


Figura 11: Teste para 1000 threads

5.3 Análise dos Resultados

Podemos observar pelos resultados obtidos acima, que o valor do consumo do CPU aumenta juntamente com número de threads e o uso de memória RAM se amntém mais ou menos constante.

Ou seja, os valores do consumo de CPU aumentam exponencialmente com o aumento do número de Threads. Em relação à memória RAM, verificamos que se mantém na gama de valores 9%-11%.

Em suma, os resultados obtidos fizeram nos perceber que a partir de um certo número de utilizadores os recursos disponíveis não seriam suficientes (consumo do CPU), levando a sobrecarga, não conseguindo cumprir com as suas funcionalidades.

6 Conclusão

Após a conclusão deste trabalho prático, atribuímos um balanço positivo ao trabalho desenvolvido, já que fizemos uso dos conceitos abordados e do trabalho realizado durante as aulas práticas.

Com o desenrolar do projeto deparamo-nos com algumas dificuldades, que fomos ultrapassando, com maior ou menor facilidade, ao nível da utilização da base de dados da GCP e da instalação de algumas ferramentas. Gostaríamos, também, de ter utilizado replicação do Wiki.js, como solução caso ocorra uma falha.

Assim, consideramos que foi alcançada uma melhor consolidação da matéria lecionada nas aulas, permitindo, ainda, a exploração de novos domínios necessários à concretização do nosso projeto.